CLEAR Agent Layer

Project status, file guide, tests, and remaining work

A plain-language guide to the AML transaction-scoring model and the connected agent-review code.

Current stage: the prepared-data-to-agent review flow is implemented and tested with mocked language-model responses. Real case-history retrieval, regulations, source-level citation checks, and the analyst application remain to be built.

Prepared 8 November 2026

1. Where the project is now

There are two model layers. The transaction ML model ranks transactions and selects alerts. The LLM review layer receives a selected alert and the evidence provided for it, then writes prosecution-side and defense-side reports. Neither score nor report decides whether laundering occurred.

- **Transaction scorer:** the modeling notebook selects/trains the scorer and saves `alerts_model/models/txn_scorer.joblib`. Prepared features and metadata are in `alerts_model/data/processed/features/`.
- **Agent service:** `agents/service.py` finds a prepared row, scores its entire day to respect the daily top-k alert budget, computes reasons for the chosen row, makes a case context, and calls the orchestrator.
- **Report validation:** LLM output must match the report format; cited IDs must be among the evidence supplied for the case.
- **Tests:** use a fake scorer and fake LLM responses. They check code flow and formatting, not real provider quality or legal correctness.

Not yet a true debate: prosecution and defense currently produce separate reports from the same evidence. The orchestrator does not pass one side's report to the other for rebuttal. The service does not fetch prior transactions related to the alert by itself.

2. How the files connect

`contracts.py` provides shared types and validation used by reports. `config.py` separately provides provider, model, and API-key settings to `llm.py`.

3. What each current file does

service.py | Class: AlertReview | Functions: review_alert, _get_modeling_module, _find_alert, _load_day

Starts review for a transaction row ID. Locates feature data, loads scorer, scores the full day for correct daily alert selection, gets the chosen row's reasons, builds context, calls review. Returns alert ID, score, reasons, and both reports.

case_context.py | Class: CaseContext | Main function: build_case_context

Packages selected alert facts, caller-supplied transaction details, and model output as evidence with stable IDs. Excludes training labels. Related transactions default to empty until a loader supplies them. Private helpers format values and make stable IDs.

orchestrator.py | Class: DebateResult | Function: review_case

Runs both agent report functions on the same context, validates citations, and returns both. Collects two views; does not yet conduct rebuttals.

prosecution.py | Function: create_report

Requires an alert flagged by the scorer, then asks the LLM to explain why supplied evidence may warrant further investigation.

defense.py | Function: create_report

Requires a flagged alert, then asks the LLM to identify limits, uncertainty, and alternative interpretations.

prompts.py | Function: build_user_prompt | Data: SYSTEM_PROMPTS

Stores side-specific instructions: use only supplied facts, cite exact IDs, do not invent facts, and do not treat alerts as proof.

llm.py | Functions: generate_report, _generate_text, _parse_report

Calls Google or Anthropic using config, converts JSON to AgentReport, rejects malformed output, then validates citations. Supports a fake text generator for tests.

contracts.py | Classes: Evidence, Finding, AgentReport | Function: validate_report

Defines evidence, claims with citations, and reports. Checks that IDs exist in supplied evidence; it does not establish truth or independently check a source.

config.py | Settings: ROOT, PROVIDER, KEY_NAME, API_KEY, MODELS, MODEL

Loads provider settings from the root .env file. Keep the actual secret in ignored .env, never in source control.

__init__.py

Marks agents as an importable Python package.

4. Tests and what their output means

From the repository root run `python -m unittest discover -s agents\tests -v`. A line ending in **ok** means one test passed. **Ran 12 tests** is the number of test methods in the current suite; **OK** means no failures. A failure shows a test name and traceback for the failed expectation.

test_contracts.py (3 tests)

Accepts a known citation; rejects missing and unknown citations.

test_case_context.py (3 tests)

Checks stable IDs, omission of training labels, a mocked prosecution report, and rejection when the row is not an alert.

test_llm.py (3 tests)

Checks valid fake JSON parsing, rejection of unknown citations, and rejection of invalid response shape.

test_orchestrator.py (1 test)

Checks both reports return and cite IDs in shared evidence.

test_service.py (2 tests)

Uses temporary Parquet samples and a fake scorer/LLM. Checks whole-day scoring then one-row explanation, and rejects a row not selected as an alert.

Tests do not contact an API and need no key. They test plumbing, not reasoning quality. A real `review_alert(row_id)` result contains `alert_id`, ML `score`, `reasons`, and `debate.prosecution`/`debate.defense`. Each report has findings and cited IDs. IDs show which supplied records the report points to; they do not prove a claim is true.

5. Components still missing

Suggested names are not fixed requirements. Coordinate with teammates responsible for data/database and the app; some components may belong outside `agents/`.

1. case_data.py or a database adapter | Next priority

Fetch the selected transaction and relevant account/counterparty history from the dataset or database. Return stable source IDs and trusted fields. Keep training labels out. Supply related rows to the service/context instead of leaving them empty.

2. regulation_retriever.py and approved source corpus

Obtain jurisdiction-specific AML laws/guidance. Preserve title, version/effective date, source URL, article/paragraph, and passage. Retrieve relevant passages as regulation evidence so agents can cite source text.

3. source_validator.py

Resolve transaction IDs to actual database records and regulation citations to exact source passages. Reject or flag missing/mismatched sources. Current code checks only membership in supplied IDs.

4. review persistence / audit integration

Store each review, both reports, cited sources, model/prompt versions, time, and human analyst actions in the project database. Keep secrets out and minimize sensitive data.

5. backend API and analyst UI

Expose an endpoint that requests a review and returns structured reports. Build alert queue, case file, verified citations, and case-specific chat. Coordinate with the proposal's database/backend and full-stack work.

6. evals/ fixtures and evaluation

Create synthetic cases and expected citation behavior. Measure citation validity, unsupported claims, usefulness of opposing views, latency, and cost for selected providers/models. Unit tests alone do not measure these.

6. Suggested order

- Implement a source-backed loader for transaction history and test it on a known synthetic case.
- Confirm jurisdiction and approved authoritative regulations with the project supervisor.
- Build regulation retrieval and source-level validation before presenting legal citations as verified.
- Add agent-quality evaluations; then run a controlled provider check on synthetic data.
- Connect the review service to the analyst backend/UI and store results for audit.

Decision-support boundary: the alert score and LLM report should assist a compliance analyst. They must not be presented as a legal finding or automatic decision.